

GramDrishti

Frontend guide: officer console, farmer app, GIS map

Build an interface that looks like a real government-agri product, not a generated template. Written for one person, from an empty folder.

You are the frontend person. You own the design system, the officer console (map, priority, review, verification, impact), the farmer mobile app, the printable bulletin, languages, and the offline demo mode.

Read in this order: Project snapshot, then Sections 1 to 12, then Appendix A (contract) and Appendix B (integration).

You never wait for the backend. From Day 2 you build on example JSON that matches the contract. Real endpoints replace them one by one.

Contents: 1 Deliverables. 2 Design direction. 3 Setup. 4 Design tokens. 5 App structure. 6 Officer screens. 7 Farmer screens. 8 Key components. 9 Data layer. 10 Languages, PWA, print. 11 Quality, accessibility, performance. 12 Timeline and polish checklist.

Project snapshot (both people read this)

GramDrishti turns a coarse block-level weather forecast into Panchayat-level forecasts with uncertainty, then into crop-stage advisories that an officer reviews before farmers receive them in Hindi, Punjabi or English. We prove the value by comparing against the plain block forecast.

Who owns what

	Backend person	Frontend person
Owens	Data, features, models, uncertainty, advisory rules, database, verification, API	Officer console, farmer app, map, charts, bulletin print page, design system, languages
Delivers	A running API at /api/v1 plus example JSON files that match the contract	A running web app that works on example JSON first, then on the real API
Never touches	React code, CSS, map styling	Model code, rules, database
Owens together	The API contract (Appendix A), integration (Appendix B), demo script	

Repository layout (one shared Git repo)

```
gramdrishti/  
  contract/      <- SHARED. openapi.json, examples/*.json, CHANGELOG.md  
  data/          <- mock data from the zip (panchayats_static.csv, etc.)  
  backend/       <- backend person only  
  frontend/      <- frontend person only  
  docs/          <- this PDF set, model card, demo script  
  docker-compose.yml <- added at integration time  
  README.md
```

Ground rules

- **Contract first.** The JSON shapes in Appendix A are frozen on Day 2. After that, any change goes through a pull request that both people approve and that adds a line to contract/CHANGELOG.md.
- **Honesty on screen.** All current data is synthetic. Every API response carries data_mode (mock or real). When it is mock, the UI shows a visible ribbon: "Synthetic demo data".
- **No invented accuracy.** Metrics shown in the UI come only from the verification job. Fixtures may hold obviously fake numbers, always with data_mode: mock.
- **Thresholds are placeholders** until a KVK or agriculture-university expert reviews them. Advisory cards carry thresholds_status, and the UI shows a small "pending expert review" label when it is placeholder.
- **Conventions.** JSON keys in snake_case. Dates as ISO YYYY-MM-DD. Coordinates as [longitude, latitude] in GeoJSON. Probabilities 0 to 1. Units fixed per variable (rain mm, temperature C, humidity %, wind km/h).
- **Working rhythm.** 15-minute sync every day. One integration branch. Merge to main only when both sides run.

Time plan assumes about 4 weeks of part-time work. If you have fewer weeks, cut the items marked "extra" first, never the core loop: forecast -> uncertainty -> advisory -> review -> verification.

1. Your deliverables and definition of done

Deliverable	Definition of done
Design system	Tokens (colour, type, spacing), 12 reusable components, documented in docs/design.md
Officer console	Map explorer, priority list, advisory review, verification, impact. Works at 1366 x 768 and larger
Farmer app	Mobile-first PWA at 360 px width: Today, Forecast, My farm. Language toggle, audio, feedback tap
Bulletin	One A4 printable Panchayat bulletin in the chosen language using print CSS
Mock API	Static JSON served from public/mock/ and switchable per endpoint
Integration	Typed API client generated from openapi.json; real endpoints switched on one at a time
Offline snapshot	App runs with no server using exported snapshot JSON
States	Loading, empty, error, and mock-data ribbon on every screen

2. Design direction

2.1 Who uses it, and what they are doing

User	Setting	Main job	Design consequence
Agromet or KVK officer	Office desktop or laptop, long sessions	Scan risk, review advisories, check the forecast is trustworthy	Dense but calm. Map is the main surface. Tables and numbers are welcome. Keyboard friendly
District or block official	Meeting room screen, occasional use	See which Panchayats need attention	Big clear map, ranked list, printable output
Farmer	Cheap Android phone in sun, patchy network, often low literacy	Know what to do today and tomorrow	One action first. Big type. Icons plus words. Voice. Works offline

2.2 The look, in one paragraph

Think of tools that people trust in the field: weather-service maps, survey and land-record portals, hydrology dashboards. They are plain, legible and information-first. GramDrishti should feel like a **well-made public service tool**: white and cool-grey surfaces, one deep green as the identity colour, standard weather colour scales on the map, and IMD-style four-level warning colours for risk. The map is the hero. There are no decorative illustrations.

2.3 Palette (use these tokens, not ad-hoc colours)

Token	Hex	Use
--ink	#18262B	Text, strongest lines
--canvas	#EEF1F0	App background (cool grey-green)
--surface	#FFFFFF	Panels, tables
--line	#D3DAD7	Borders, dividers
--muted	#5B6B70	Secondary text (checked for contrast on white)
--brand	#24594A	Primary buttons, selected nav, links
--water	#2A6FA8	Rain and water values
--heat	#C4551D	Temperature highlights
--sev-low / mod / high / severe	#3E8E41 / #E8B923 / #E0801F / #C62828	Risk levels. Always paired with a text label, never colour alone

No gradients, no glass blur, no purple or indigo, no neon. Maps use established scientific scales: Blues for rain, YlOrRd for temperature, GnBu for humidity, Greys for wind, and a red-blue diverging scale for "difference from block".

2.4 Typography

- **Latin and numbers:** Source Sans 3 (free, Google Fonts). It reads well at small sizes in tables. Use `font-variant-numeric: tabular-nums` for all numbers.
- **Hindi:** Noto Sans Devanagari. **Punjabi:** Noto Sans Gurmukhi. Both are free. Self-host the font files so the demo works offline.
- **Scale:** 12 / 14 / 16 / 20 / 28 px on desktop. Farmer app body text starts at 17 px, action headline 24 px.
- Sentence case everywhere. No all-caps labels, no letter-spaced tags above headings.
- Line length under 75 characters in reading text. Devanagari and Gurmukhi need about 15 to 20 percent more line height.

2.5 Layout rules

- **Officer console:** left icon-plus-label navigation (Map, Priority, Review, Verification, Impact), a top bar with district, issue date and role, and the map opens immediately. The right side panel holds details of the selected Panchayat. This is the standard layout of GIS tools, which is exactly why it feels real.
- **No row of identical KPI cards** on top of the page. If a number matters, place it where it is used, for example next to the map legend.
- **Radius 3 px** for inputs and buttons, 0 to 4 px for panels. Shadows only for popovers. Borders do the separating work.
- **Density:** table rows 36 px, 14 px text. Officers scan lists.
- **Farmer app:** single column, 48 px minimum touch targets, bottom navigation with three items, one primary action per screen.

2.6 What makes generated UIs look generated, and what to do instead

Generated-looking habit	Do this instead
Gradient hero and a big number with a coloured badge	Open on the working map. Put real values in context
Rows of identical rounded cards with soft shadows	Use a table, a list with dividers, or one panel. Vary structure by content
Emoji as icons	One consistent line-icon set (Lucide) plus small custom crop glyphs
Purple/indigo accent, cream + terracotta, or black + neon	The green, blue and grey tokens above
ALL-CAPS tiny labels above every heading	Normal headings only where they help navigation
Fade-and-slide on every section	No entrance animation. Motion only for responses to user actions (panel opens, toggle switches)
Lorem ipsum and fake names like John Doe	Use real fields from the data: Synthetic Panchayat MP0103, real crops, real dates, real units
Vague copy: 'Insights', 'Empower', 'Smart'	Say what it is: 'Rain chance tomorrow', 'Needs attention', 'Waiting for review'
Everything centred	Left-aligned text and numbers right-aligned in tables

2.7 Voice and microcopy

Place	Write	Avoid
Advisory title	Irrigate tomorrow morning	Optimised irrigation recommendation
Confidence	Likely / Possible / Uncertain	Confidence score 0.73
Uncertainty	Rain amount likely between 2 and 14 mm	P10 P90
Empty list	No advisories waiting for review for 16 Dec. Change the date or run the daily job.	Nothing here yet!
Error	Could not load the forecast for MP0103. Check the server is running, then try again.	Oops, something went wrong
Buttons	Approve, Edit wording, Reject, Listen, Print bulletin	Submit, Go, Continue
Mock ribbon	Synthetic demo data. Not real weather.	Demo mode

Keep one word per action across the whole flow: the button says **Approve**, the toast says **Approved**, the audit log says **approved**.

3. Setup from scratch

3.1 Install

- Node.js 20 LTS, Git, VS Code with ESLint and Prettier extensions, Chrome for device emulation.

3.2 Create the app

```
cd gramdrishti
npm create vite@latest frontend -- --template react-ts
cd frontend && npm install
npm install react-router-dom @tanstack/react-query maplibre-gl recharts i18next react-i18next
  lucide-react zustand
npm install -D openapi-typescript vite-plugin-pwa vitest msw @types/node
mkdir -p src/{api,components,features,pages,i18n,styles,lib} public/mock public/fonts
```

3.3 Scripts in package.json

```
"scripts": {
  "dev": "vite",
  "build": "tsc -b && vite build",
  "gen:types": "openapi-typescript ../contract/openapi.json -o src/api/schema.d.ts",
  "test": "vitest",
  "preview": "vite preview"
}
```

3.4 Folder map

Folder	Contents
src/api/	client.ts, hooks.ts, schema.d.ts (generated), mock.ts
src/styles/	tokens.css, base.css, print.css
src/components/	Reusable UI: Ribbon, RiskChip, ConfidenceLabel, AdvisoryCard, FanChart, MapView, Legend, DataTable, LangSwitch, AudioButton
src/features/	Screen logic: map/, priority/, review/, verification/, impact/, farmer/, bulletin/
src/pages/	Route components that assemble features
src/i18n/	en.json, hi.json, pa.json
public/mock/	Copies of contract/examples/*.json, plus panchayats.geojson, blocks.geojson

4. Design tokens (write these first)

```
/* src/styles/tokens.css */
:root {
  --ink:#18262B; --canvas:#EEF1F0; --surface:#FFFFFF; --line:#D3DAD7; --muted:#5B6B70;
  --brand:#24594A; --brand-ink:#FFFFFF; --water:#2A6FA8; --heat:#C4551D;
  --sev-low:#3E8E41; --sev-mod:#E8B923; --sev-high:#E0801F; --sev-severe:#C62828;
  --radius:3px; --gap-1:4px; --gap-2:8px; --gap-3:12px; --gap-4:16px; --gap-5:24px;
  --font: "Source Sans 3", "Noto Sans Devanagari", "Noto Sans Gurmukhi", system-ui, sans-serif;
}
html { font-family: var(--font); color: var(--ink); background: var(--canvas); font-size: 16px; }
body { margin: 0; font-variant-numeric: tabular-nums; }
:lang(hi) { line-height: 1.65; } :lang(pa) { line-height: 1.65; }
:focus-visible { outline: 2px solid var(--brand); outline-offset: 2px; }
@media (prefers-reduced-motion: reduce) { * { animation: none !important; transition: none !important; } }
```

Fonts: download Source Sans 3, Noto Sans Devanagari and Noto Sans Gurmukhi (regular, medium, bold) into public/fonts/ and declare them with @font-face and font-display: swap. Check each licence file is included in the repository.

5. App structure

Route	Screen	Audience
/	Redirects to /map	officer
/map	Map explorer with detail panel	officer
/priority	Panchayats needing attention	officer
/review	Advisory review queue and editor	officer
/verification	Forecast accuracy check	officer, jury
/impact	Decision replay	officer, jury
/farmer	Farmer app (Today, Forecast, My farm inside)	farmer (opens at phone width)
/bulletin/:pid	Printable A4 Panchayat bulletin	sarpanch, officer

- **Global state (zustand):** issueDate, leadDay, variable, viewMode (block | panchayat | delta), selectedPid, lang, role. Mirror issueDate, variable and selectedPid into the URL so a link reproduces the screen.

- **Server state (TanStack Query):** every API call is a hook. Do not copy server data into zustand.
- **Role switch:** a small control in the top bar (Officer / Farmer). It only changes the layout. State clearly that there is no login in the prototype.
- **Mock ribbon:** a 28 px strip under the top bar, shown whenever `meta.data_mode === 'mock': "Synthetic demo data. Not real weather."`

6. Officer console screens

6.1 Map explorer (the main screen)

GramDrishti		Synthetic District v		Issued 12 Aug 2024 v		Officer Farmer	
Synthetic demo data. Not real weather.							
Map	Day [1][2][3][4][5]					MP0103	
Prio	Show: Rain Tmax Tmin					Block MB01	
Rev	Humidity Wind					Rain tomorrow	
Ver				M A P		9.8 mm (0 to 31)	
Imp	View:			Panchayat polygons coloured,		Block: 12.4 mm	
	() Block			block outlines dark		[5-day fan chart]	
	(x) Panchayat					Why different?	
	() Difference					- low-lying land	
	Risk layer: none v					- clay soil	
				legend bottom-left		Advisories (2)	

- **View modes:** Block paints every Panchayat with its block value (flat colour per block). Panchayat paints each value. Difference paints Panchayat minus block on a diverging scale. Toggling between them is the single most convincing moment of the demo, so make it instant.
- **Lead-day buttons** show the date: Tue 13 Aug, not Day 1.
- **Hover** a Panchayat: tooltip with name, p50 and range. **Click:** opens the right panel and updates the URL.
- **Right panel content, top to bottom:** header with name and block; tomorrow's value with a plain-language range; the fan chart for the chosen variable; a small table of all variables for the chosen day; "Why is it different from the block?" (from `/explain`); "Forecast changed since yesterday" (from `/forecast/changes`); advisories for this Panchayat.
- **Observed overlay:** a toggle "Show what happened" adds observed points to the fan chart using `/observed`. Label the source (station or synthetic truth).
- **Table alternative:** a button "Show as table" lists Panchayats with the same values. Needed for accessibility and useful for officers.

6.2 Priority

Map	Panchayats needing attention		Next [2 days v]		Type [All v]	Block [All v]	
Prio	#	Panchayat	Block	Main risk	Level	Crops	Why
Rev	1	MP0103	MB01	Waterlogging	High	Paddy	Heavy rain, poor drainage
Ver	2	MP0412	MB04	Heat	Moderate	Wheat	Tmax near grain-fill limit
Imp	3	MP0207	MB02	Heavy rain	Moderate	Cotton	Over 35 mm possible Thu

- Level chip = colour + word (for example an orange chip that says High). Sort by score by default, allow column sorting.
- Row click goes to `/map?pid=...` Button **Print list** uses print CSS.

6.3 Review queue

Map	Waiting for review (14)	Irrigation	Wheat, jointing	MP0103
Prio	[status v] [block v] [crop v]	Valid 17-19 Dec	Possible	Thresholds: placeholder
Rev	> MP0103 wheat irrigation	English	Hindi	Punjabi (tabs)
Ver	MP0207 paddy spray hold	Action:	[editable text area]	
Imp	MP0412 wheat heat	Reason:	[editable text area]	
		Evidence:	Rain over 5 mm, next 3 days	18%
			Soil water left	41%
		[Approve]	[Save edit and approve]	[Reject] [Listen]
		History: created by system 08:02 ...		

- Keyboard: arrow keys move through the list, Enter opens, Ctrl+Enter approves.
- After a review call, invalidate the advisories and farmerAdvice queries so the farmer screen updates.
- Show the audit trail under the editor. It is a trust feature for the jury.
- Show a small line "Thresholds pending expert review" when thresholds_status is placeholder.

6.4 Verification

This screen answers the jury's question. Keep it sober and honest.

- **Top:** one sentence built from /verification/summary: what data, what period, which method. Include the mock notice when data_mode is mock.
- **Table:** rows = variable and metric; columns = Model, Block forecast (B0), Corrected block (B1), Simple physical (B2), Skill vs B0 with its 95% interval. Bold the best value per row. Show rows where the model does **not** win, unhidden.
- **Reliability plot** for the chosen rain event: forecast probability on x, observed frequency on y, the diagonal as the perfect line, and point size by sample count.
- **Coverage chart:** nominal 80% versus empirical coverage per variable.
- **Where it works:** a small table from /verification/regions (held-out block or station, error versus B0).
- **Footnote:** "Synthetic proxy validation. Real-station validation follows when data is available." The wording comes from notes in the API.

6.5 Impact

- Choose a decision (spray, heat alert, irrigation wait). Show two horizontal stacked bars, Model and Block forecast, with three parts: correct, wasted wait, washed off. Numbers written on the bars.
- A line under the bars explains the rule in plain words, for example "Advise spraying when the chance of 2.5 mm rain is under 30%".
- No money values. Counts and percentages only.

7. Farmer app screens (mobile first, 360 px wide)

Village MP0103	< Forecast	My farm
Hindi Punjabi English	Tue 13 Wed 14 Thu 15	
Today	Rain Likely 0 to 31 mm	Panchayat MP0103
[Icon] Irrigate tomorrow	Temp 22 to 34 deg C	Language Hindi
morning	Wind 7 km/h	Wheat, sown 12 Nov
Soil is drying and rain	Spray Good Caution	Paddy, sown 1 Jul
is unlikely before Fri	[5-day strip]	[Add crop]
Likely	Did it rain today?	Did it rain today?
[Listen] [Share]	[Yes] [No]	[Yes] [No]
Also today (2)		
> Do not spray on Wed		
> Bulletin [Print]		
[Today]	[Forecast]	[My farm]

<- bottom navigation, always visible

- **Today** shows the single highest-priority approved advisory as a full-width block: icon, action in 24 px, one-line reason, confidence word, then Listen and Share. Other advisories are a plain list below.
- **Forecast** shows a 5-day strip with three rows for each day: rain (range in words and mm), temperature, wind. Spray suitability (Good, Caution, Avoid) uses icon plus word.
- **My farm** holds Panchayat, language, crops with sowing dates (from `/farmers/{id}`). Editing can be mocked in the prototype (local state only). It also holds the feedback control.
- **Feedback:** "Did it rain today?" with Yes and No buttons. After Yes: light, moderate, heavy. Post to `/feedback`. Show "Thank you. Your report helps improve the forecast for your village."
- **Share:** builds a WhatsApp text (`https://wa.me/?text=...`) with the advisory in the current language. No contact numbers stored.
- **Audio:** Listen plays `/audio/{id}?lang=` if available. If it returns 404 or the device is offline, fall back to `speechSynthesis` with the language tag (`hi-IN`, `pa-IN`, `en-IN`). Test it on a real phone because voices differ.
- **Offline:** last fetched advisories and forecast are cached by the service worker and shown with a small "Last updated 16 Dec, 08:10" line.

7.1 Bulletin (print page)

- Route `/bulletin/:pid?lang=hi`. A4 portrait, black and white safe, 12 pt text minimum. Contents: Panchayat name, date, 3-day summary strip, up to 4 actions with reasons, one line on data status, and a note that advisories are reviewed by an officer.
- Use `@media print` to hide the navigation and ribbon, set `@page { size: A4; margin: 14mm; }`, and avoid backgrounds. The browser's Save as PDF is the export.
- Show the mock notice in the print footer when data is synthetic.

8. Key components

Component	Behaviour	Notes
MapView	MapLibre GL map showing GeoJSON polygons coloured by feature-state	No basemap required. Optional light raster basemap with attribution. Fit bounds to district on load
Legend	Colour ramp with units and break values	Same breaks as the map style, generated from one config
FanChart	5-day p10 to p90 band, p50 line, dashed block line, optional observed points	Recharts <code>ComposedChart</code> . Band drawn as two stacked areas (p10 transparent, p90 minus p10 filled)
RiskChip	Coloured chip with text: Low, Moderate, High, Severe	Colour and word always together
ConfidenceLabel	Likely / Possible / Uncertain	Maps high/medium/low. Tooltip explains
AdvisoryCard	Title, action, reason, evidence rows, time window, fallback, confidence, buttons	Two variants: compact (list) and hero (Today)
ExplainList	Top reasons from <code>/explain</code>	Plain sentences with a small up/down icon
DiffEditor	Three language tabs with text areas and a changed-since-draft marker	Highlights edited fields
ReliabilityPlot	Scatter with diagonal	SVG or Recharts <code>ScatterChart</code>
DataTable	Sortable, sticky header, 36 px rows	Use for priority, verification and station lists
LangSwitch	Shows each language in its own script (Hindi, Punjabi, English)	Stores choice in <code>localStorage</code>
AudioButton	API audio with <code>speechSynthesis</code> fallback	Shows playing state
Ribbon	Mock data notice	Not dismissible

8.1 Map colouring with feature-state

```
// src/components/MapView.tsx (core idea)
map.addSource("gp", { type: "geojson", data: "/mock/panchayats.geojson", promoteId: "panchayat_id" });
map.addSource("blk", { type: "geojson", data: "/mock/blocks.geojson" });
map.addLayer({ id: "gp-fill", type: "fill", source: "gp", paint: {
  "fill-color": ["case", ["==", ["feature-state", "v"], null], "#E3E8E6",
    ["interpolate", ["linear"], ["feature-state", "v"], ...rampStops]], // e.g. Blues
    for rain
  "fill-opacity": 0.9 }});
map.addLayer({ id: "gp-line", type: "line", source: "gp", paint: { "line-color": "#FFFFFF",
  "line-width": 0.6 }});
map.addLayer({ id: "blk-line", type: "line", source: "blk", paint: { "line-color": "#18262B",
  "line-width": 1.6 }});

// when data or view mode changes: one loop, no re-adding layers
for (const r of panchayatLayer) {
  const v = mode === "block" ? r.block_value : mode === "delta" ? r.delta : r.p50;
  map.setFeatureState({ source: "gp", id: r.panchayat_id }, { v });
}
```

8.2 Colour ramps (one config file lib/ramps.ts)

Variable	Ramp	Breaks
Rain (mm)	Blues (ColorBrewer)	0, 1, 5, 15, 35, 65
Tmax / Tmin (C)	YlOrRd for Tmax; YlGnBu reversed for Tmin	Breaks each 3 C over the seasonal range
Humidity (%)	GnBu	20, 40, 60, 80, 100
Wind (km/h)	Greys	0, 5, 10, 15, 20, 30
Difference	RdBu diverging, centred on 0	Symmetric around zero, range from data
Risk	The four severity colours	Low, Moderate, High, Severe

8.3 Fan chart data shaping

```
const rows = days.map(d => ({
  date: fmtDay(d.date),
  p10: d[v].p10, base: d[v].p10, band: (d[v].p90 ?? 0) - (d[v].p10 ?? 0),
  p50: d[v].p50, block: d[v].block, observed: obs[d.date] ?? null,
}));
// <Area dataKey="base" stackId="s" stroke="none" fill="transparent" />
// <Area dataKey="band" stackId="s" stroke="none" fill="var(--water)" fillOpacity={0.25} />
// <Line dataKey="p50" stroke="var(--water)" /> <Line dataKey="block" strokeDasharray="4 3"
//   stroke="var(--ink)" />
// <Scatter dataKey="observed" /> (only when the observed toggle is on)
```

9. Data layer

9.1 API client with per-endpoint mock switch

```
// src/api/client.ts
const BASE = import.meta.env.VITE_API_BASE ?? "/api/v1";
const REAL = (import.meta.env.VITE_REAL_ENDPOINTS ?? "").split(","); // e.g.
  "meta,geo,forecast"
const SNAP = import.meta.env.VITE_SNAPSHOT === "1";

export async function get<T>(group: string, path: string, mockFile: string): Promise<T> {
  const useReal = !SNAP && REAL.includes(group);
  const url = useReal ? BASE + path : "/mock/" + mockFile; // mock file names match
    the examples
  const res = await fetch(url);
  if (!res.ok) throw new ApiError(res.status, await res.json().catch(() => null));
  return res.json();
}

// src/api/hooks.ts
export const usePanchayat = (pid?: string, issue?: string) =>
  useQuery({ queryKey: ["panchayat", pid, issue], enabled: !!pid && !!issue,
    queryFn: () => get<PanchayatForecast>("forecast",
      `/forecast/panchayat/${pid}?issue_date=${issue}`,
      `forecast_panchayat_${pid}.json`) });
```

- Types come from `src/api/schema.d.ts`. Example: `type PanchayatForecast = components['schemas']['PanchayatForecast']`.
- Set `staleTime` to a few minutes for forecasts and 0 for the review queue.
- **Every hook returns loading, error and empty states** that the screen renders. Write one `<QueryBoundary>` component to keep this consistent.
- After `POST /advisories/{id}/review` call `queryClient.invalidateQueries({ queryKey: ['advisories'] })` and the same for `farmerAdvice`.
- Treat null values as missing: skip them in charts, show a dash in tables.

9.2 Mock files

The backend person gives you `contract/examples/`. Copy or symlink them into `public/mock/`. Missing example for a screen? Write a small one by hand from Appendix A and tell the backend person, so it becomes part of the contract.

With MSW you can also simulate slow responses and errors to see your loading and error states. Do this once for each screen.

10. Languages, PWA and print

10.1 Languages

- Interface strings live in `en.json`, `hi.json`, `pa.json`. Advisory text arrives already translated from the API (`action.hi`, and so on). Do not translate advisories on the client.
- Use `i18next` with keys grouped by screen. Keep a test that fails if any key exists in English but not in Hindi or Punjabi.
- Set `document.documentElement.lang` when the language changes so fonts and line height switch.
- Dates and numbers: `Intl.DateTimeFormat` and `Intl.NumberFormat` with `hi-IN` and `pa-IN`. Keep numerals consistent across the screen.
- Leave 30 to 40 percent extra width for buttons in Hindi and Punjabi. Test every screen in all three languages.
- Ask a native speaker to check UI strings. Backend handles advisory text review.

10.2 PWA and offline

- Configure `vite-plugin-pwa` with a manifest (name, short name, theme colour #24594A, icons) and `registerType: 'autoUpdate'`.
- Cache the app shell, fonts and `/mock` files. Cache API GET responses for forecast and advisories with a network-first strategy and a fallback to cache.
- Show an offline pill in the top bar and the "Last updated" line when data come from cache.
- Snapshot mode (`VITE_SNAPSHOT=1`) reads only from `public/snapshot/` exported by the backend. It is your demo insurance.

10.3 Print CSS

```
/* src/styles/print.css */
@media print {
  nav, .ribbon-top, .toolbar, button { display: none !important; }
  body { background: #fff; color: #000; font-size: 12pt; }
  @page { size: A4; margin: 14mm; }
  .bulletin-day { break-inside: avoid; }
}
```

11. Quality, accessibility and performance

11.1 Accessibility checklist

- Text contrast at least 4.5 to 1 on all surfaces (check - -muted on white and on - -canvas).
- Severity, confidence and status are never colour-only: chip text or icon always present.
- Everything reachable and operable by keyboard, with the visible focus style from the tokens.
- Map has a table alternative and each Panchayat has an accessible name.
- Buttons at least 44 px high on mobile. Inputs have visible labels.
- prefers-reduced-motion respected (already in tokens).
- Lang attribute set for Hindi and Punjabi text so screen readers pronounce correctly.

11.2 Performance

- The GeoJSON for 90 Panchayats is small. For state scale you would move to vector tiles. Say so in the presentation.
- Update the map with `setFeatureState`, not by rebuilding layers.
- Code-split routes (`React.lazy`) so the farmer app loads without the officer console.
- Target a usable first screen on a throttled 'Fast 3G' profile in Chrome DevTools for the farmer app.

11.3 Testing

- **Vitest:** ramp functions, date formatting, confidence mapping, i18n key parity.
- **Schema check:** a script that loads each `public/mock/*.json` and validates it against the generated types. It catches contract drift early.
- **Manual device test:** Chrome device emulation at 360 x 640, plus one real low-end phone if possible.
- **Screenshot pass:** take screenshots of every screen in all three languages and read them at full size once before the demo.

12. Timeline and final polish

Days	Tasks	You can show
1 - 2	Repo, Vite app, tokens, fonts, app shell with top bar, nav and ribbon, mock loader, ramps config	Empty shell in brand style
3 - 5	Map explorer with block, Panchayat and difference modes; legend; lead-day buttons	Flat block colour turning into Panchayat variation
6 - 8	Detail panel, fan chart, table of variables, observed overlay, explain list	One Panchayat fully explained
9 - 11	Priority screen, risk layer, review queue with editor and audit trail	Approve flow
12 - 14	Farmer app: Today, Forecast, My farm, feedback, audio, share	Phone-width demo in Hindi
15 - 16	Verification and impact screens	Reliability plot and comparison table
17 - 18	Bulletin print page, PWA, offline snapshot mode	Print to PDF, airplane-mode demo
19 - 20	Accessibility pass, language pass, loading and error states, polish	Rehearsal ready

Final polish checklist

Check	Done
No emoji, no gradients, no purple, no decorative illustrations	
Every number has a unit; every date is human readable (Tue 13 Aug)	
Mock ribbon appears on every screen and on print	
All copy in sentence case; button and toast vocabulary matches	
Loading, empty and error states exist and read like real product text	
Hindi and Punjabi have no missing keys, no overflow, no font boxes	
Farmer app works at 360 px with one thumb	
Officer app looks right at 1366 x 768 and on a projector	
Approve on Review changes the farmer Today screen	
Verification screen shows losses as clearly as wins	
Snapshot mode runs the whole demo with the server stopped	

Appendix A. API contract v0.1 (shared, frozen Day 2)

This is the handshake between the two of you. The backend implements it. The frontend builds against it. Both sides write their tests from the same examples. Base URL in development: `http://localhost:8000/api/v1`.

A1. Conventions and enums

Item	Rule
IDs	panchayat_id like MP0103, block_id like MB01, station_id like MOCK_AWS_01, advisory id is a readable string
Variables (var)	rain (mm/day), tmax (C), tmin (C), rh (% daily mean), wind (km/h)
Quantile object	{p10, p50, p90, block} where block is the block-level value for the same day. Rain p10 can be 0.
Risk type	heavy_rain, heat, frost, waterlogging, dry_spell
Level	low, moderate, high, severe (UI colours: green, yellow, orange, red as in IMD warning colours)
Advisory category	sowing, irrigation, spray, fertilizer, harvest, heat_stress, frost, waterlogging, dry_spell, pest_disease, livestock
Status	draft, approved, edited, rejected
Confidence	high, medium, low
Language	en, hi, pa
Nulls	Missing numbers are null, never NaN, never the string "NaN"
Errors	HTTP 4xx/5xx with body {"error": {"code": "not_found", "message": "Panchayat MP9999 does not exist"}}
Versioning	GET /meta returns api_version. Contract changes bump the minor version.
Auth	None in the prototype. Header X-Role: officer or farmer is sent by the UI for logging only. State this honestly in the demo.

A2. Endpoints

Method + path	Purpose	Used by
GET /health	Liveness check	both
GET /meta	District name, data_mode, api_version, available_issue_dates, languages, crops	all screens
GET /geo/panchayats	GeoJSON polygons (properties: panchayat_id, name, block_id)	map
GET /geo/blocks	GeoJSON block polygons	map
GET /forecast/map?issue_date&lead_day&var	Values for both map modes: block layer and Panchayat layer	map
GET /forecast/panchayat/{id}?issue_date	5-day detail with quantiles, event probabilities, derived agro-variables	detail panel, farmer app
GET /observed/panchayat/{id}?from&to	What actually happened (station or synthetic truth, with source)	forecast vs observed overlay
GET /explain/{id}?issue_date&lead_day&var	Top 3 plain-language reasons this Panchayat differs from its block	detail panel
GET /risk?issue_date&lead_day&type	Risk level and score per Panchayat for one risk type	risk map
GET /priority?issue_date&horizon_days	Ranked list of Panchayats needing attention	priority screen
GET /advisories?status&panchayat_id&issue_date	Advisory list for the review queue	review screen
GET /advisories/{id}	One advisory with evidence and audit trail	review screen
POST /advisories/{id}/review	Approve, edit or reject (writes audit log)	review screen
GET /audio/{advisory_id}?lang	MP3 of the advisory text (404 if not available)	farmer app
GET /farmers/{id}	Demo farmer profile: Panchayat, crops, sowing dates, language	farmer app
GET /farmers/{id}/advice?issue_date	Approved advisories relevant to this farmer	farmer app
POST /feedback	Farmer taps "Did it rain?"	farmer app
GET /forecast/changes/{id}?issue_date	What changed since the previous forecast and whether advice changed	farmer app, detail panel
GET /verification/summary	Model vs baselines B0, B1, B2 per variable, with confidence intervals	verification screen
GET /verification/reliability?event	Reliability diagram points for one event	verification screen
GET /verification/coverage	Nominal vs empirical interval coverage	verification screen
GET /verification/regions	Error by held-out block or station	verification screen
GET /impact?season	Replay of past advice: model vs block-level decisions	impact screen
GET /data-quality	Stations reporting, missing share, stale inputs	officer console

A3. Example payloads

Numbers below are structural examples only. The backend generates real ones from the mock data in `contract/examples/`.

GET /meta

```
{ "api_version": "0.1", "data_mode": "mock", "district": "Synthetic District",
  "available_issue_dates": ["2024-08-12", "2024-08-13", "2024-12-16"],
  "languages": ["en", "hi", "pa"], "crops": ["wheat", "paddy", "cotton", "mustard", "bajra"],
  "vars": [ {"var": "rain", "unit": "mm"}, {"var": "tmax", "unit": "C"}, {"var": "tmin", "unit": "C"},
    {"var": "rh", "unit": "%"}, {"var": "wind", "unit": "km/h"} ] }
```

GET /forecast/map?issue_date=2024-08-12&lead_day=1&var=rain

```
{ "issue_date": "2024-08-12", "valid_date": "2024-08-13", "lead_day": 1, "var": "rain", "unit": "mm",
  "block_layer": [ {"block_id": "MB01", "value": 12.4} ],
  "panchayat_layer": [ {"panchayat_id": "MP0103", "block_id": "MB01", "p10": 0.0, "p50": 9.8, "p90":
    31.0,
    "block_value": 12.4, "delta": -2.6, "prob_event": 0.71, "event":
      "rain_ge_2_5mm"} ] }
```

GET /forecast/panchayat/MP0103?issue_date=2024-08-12

```
{ "panchayat_id": "MP0103", "name": "Synthetic Panchayat MP0103", "block_id": "MB01",
  "issue_date": "2024-08-12", "data_mode": "mock",
  "static": { "elevation_m": 214.8, "soil_texture": "clay_loam", "drainage_class": "poor",
    "irrigated_frac": 0.62 },
  "days": [ {
    "date": "2024-08-13", "lead_day": 1,
    "rain": { "p10": 0.0, "p50": 9.8, "p90": 31.0, "block": 12.4},
    "tmax": { "p10": 32.1, "p50": 33.4, "p90": 35.0, "block": 33.9},
    "tmin": { "p10": 24.0, "p50": 25.1, "p90": 26.2, "block": 25.3},
    "rh": { "p10": 66, "p50": 74, "p90": 82, "block": 72},
    "wind": { "p10": 4.1, "p50": 7.3, "p90": 11.0, "block": 7.9},
    "prob": { "rain_ge_1mm": 0.78, "rain_ge_2_5mm": 0.71, "rain_ge_10mm": 0.42, "rain_ge_35mm": 0.06},
    "derived": { "et0_mm": 4.6, "soil_moisture_frac": 0.81, "thi": 78.2, "waterlog_risk": "moderate"} }
  ] }
```

GET /advisories/{id}

```
{ "id": "ADV-2024-12-16-MP0103-wheat-irrigation", "issue_date": "2024-12-16",
  "panchayat_id": "MP0103", "block_id": "MB01", "crop": "wheat", "stage": "jointing_booting",
  "category": "irrigation", "priority": "high", "valid_from": "2024-12-17", "valid_to": "2024-12-19",
  "action": { "en": "Irrigate tomorrow morning. Skip if you already irrigated in the last 3 days.",
    "hi": "...", "pa": "..."},
  "reason": { "en": "Soil is drying and rain is unlikely before Friday.", "hi": "...", "pa": "..."},
  "fallback": { "en": "If rain above 5 mm arrives overnight, skip irrigation.", "hi": "...", "pa":
    "..."},
  "confidence": "medium",
  "evidence": [ {"label": "Chance of rain over 5 mm, next 3 days", "value": "18%"},
    {"label": "Soil water left in root zone", "value": "41%"} ],
  "thresholds_status": "placeholder",
  "status": "draft", "reviewed_by": null, "reviewed_at": null,
  "audit": [ {"at": "2024-12-16T08:02:11", "actor": "system", "action": "created", "note": ""} ],
  "audio": { "hi": "/api/v1/audio/ADV-2024-12-16-MP0103-wheat-irrigation?lang=hi" } }
```

POST /advisories/{id}/review (request) and POST /feedback (request)

```
{ "action": "edit", "reviewer": "Officer Demo", "note": "Shortened wording",
  "edited": { "action": { "en": "...", "hi": "...", "pa": "..."} } } <- action: approve | edit
| reject

{ "panchayat_id": "MP0103", "date": "2024-08-13", "reported_rain": true, "intensity": "moderate",
  "channel": "app" } <- intensity: none | light
| moderate | heavy
```

GET /priority, /explain, /verification/summary, /verification/reliability, /impact

```
/priority -> { "issue_date": "...", "items": [ { "panchayat_id": "MP0103", "block_id": "MB01",
  "top_risk": "waterlogging", "level": "high", "score": 0.74, "crops_affected": ["paddy"],
  "headline": {"en": "Heavy rain likely tomorrow on poorly drained soil"} } ] }

/explain -> { "panchayat_id": "MP0103", "var": "tmax", "delta_vs_block": -0.5,
  "reasons": [ {"feature": "irrigated_frac", "effect": "cooler", "text": {"en": "More irrigated fields
    around it"}} ] }

/verification/summary -> { "data_mode": "mock", "method": "leave_one_block_out + temporal_holdout",
  "period": {"start": "2024-07-16", "end": "2024-12-31"},
  "variables": [ { "var": "tmax", "n": <int>, "metrics": [ { "name": "MAE", "unit": "C",
    "model": <num>, "b0": <num>, "b1": <num>, "b2": <num>, "skill_vs_b0": <num>,
    "skill_ci95": [<num>, <num>] } ] } ],
  "events": [ { "event": "rain_ge_2_5mm", "pod": <num>, "far": <num>, "csi": <num>, "brier": <num>,
    "brier_skill_vs_climatology": <num> } ],
  "block_mean_error": {"rain": <num>, "tmax": <num>}, "notes": ["Proxy validation on synthetic truth"]
}

/verification/reliability?event=rain_ge_2_5mm -> { "points": [ {"forecast_prob": 0.1, "observed_freq":
  0.08, "n": 812} ] }

/impact?season=monsoon_2024 -> { "decision": "spray", "model": {"correct": <int>, "wasted_wait": <int>,
  "washed_off": <int>},
  "block_baseline": {"correct": <int>, "wasted_wait": <int>, "washed_off": <int>}, "n_decisions":
  <int> }
```

Placeholders written as <num> or <int> are filled by the verification job. Nobody types accuracy numbers by hand.

Appendix B. How the two halves are integrated

Integration is not a final-week event. You integrate in small steps, one endpoint at a time, from Week 1. The idea: the frontend never waits for the backend, and the backend never guesses what the frontend needs.

B1. The five integration mechanisms

1. **Shared contract folder.** `contract/openapi.json` (generated by FastAPI) and `contract/examples/*.json` (hand-checked). Both people read from it.
2. **Example JSON as the first backend.** On Day 2 the backend person exports one example file per endpoint using mock data (even crude). The frontend serves them as a fake API and builds every screen from them.
3. **Generated types.** Frontend runs `npm run gen:types` (uses `openapi-typescript` on `contract/openapi.json`). If the backend changes a field, the frontend build fails immediately instead of showing a blank chart in the demo.
4. **Endpoint-by-endpoint switch.** The frontend has one env variable listing which endpoints use real data: `VITE_REAL_ENDPOINTS=meta,geo,forecast`. Everything else still reads examples. This makes progress visible and bugs local.
5. **Contract tests on both sides.** Backend tests check each real response validates against its Pydantic model and equals the example's shape. Frontend tests parse each example with its type guards.

B2. Milestones

Milestone	Backend delivers	Frontend delivers	Joint check
M0 Day 1-2	Repo, data loading, contract examples	Repo, design tokens, app shell, mock API	Both run locally. Contract frozen.
M1 end Week 1	<code>/meta</code> , <code>/geo/*</code> , <code>/forecast/map</code> (baseline B0 values are fine)	Map screen with block/Panchayat toggle	Map shows real polygons from the API
M2 mid Week 2	Model forecasts, quantiles, <code>/forecast/panchayat</code> , <code>/observed</code>	Detail panel, fan chart, observed overlay	Forecast vs observed for one Panchayat looks sensible
M3 end Week 2	Advisory engine, DB, review endpoints, <code>/priority</code> , <code>/risk</code>	Review queue, priority list, risk map, advisory card	Officer approves a card that then appears on farmer side
M4 mid Week 3	Verification, impact, explain endpoints	Verification and impact screens	Reliability plot and B0 vs model table are real
M5 end Week 3	Feedback, changes, audio, data-quality	Farmer app, language toggle, audio, bulletin print page	Full farmer journey works in three languages
M6 Week 4	Hardening, docs, snapshots	Polish, accessibility, offline snapshot	Two full dry runs of the demo script

B3. Running both together

```
# terminal 1 - backend
cd backend && source .venv/bin/activate
uvicorn gramdrishti.api.main:app --reload --port 8000

# terminal 2 - frontend
cd frontend && npm run dev # http://localhost:5173

# frontend/.env.local
VITE_API_BASE=http://localhost:8000/api/v1
VITE_REAL_ENDPOINTS=meta,geo,forecast # grow this list as endpoints go live

# export the contract after any backend API change (backend person)
python -m gramdrishti.export_openapi # writes contract/openapi.json
# regenerate frontend types (frontend person)
npm run gen:types # openapi-typescript ../contract/openapi.json -o
src/api/schema.d.ts
```

- **CORS:** the backend allows `http://localhost:5173` (and the deployed frontend origin later). If the browser shows a red CORS error, this is almost always why.
- **Docker (Week 4):** one `docker-compose.yml` with two services, `api` (port 8000) and `web` (nginx serving the built frontend, proxying `/api` to `api`). This gives you a single command for the demo laptop.
- **Demo laptop rule:** the final demo runs from the same machine setup you rehearsed on. Do not update packages the night before.

B4. Integration checklist (tick each before the demo)

Check	Owner	Done
Every example JSON validates against its Pydantic model	Backend	
<code>openapi.json</code> regenerated after last API change; types regenerated	Both	
No NaN or Infinity anywhere in real responses (search with curl)	Backend	
GeoJSON is <code>[longitude, latitude]</code> ; map renders in the right place	Both	
Dates and time zones: all ISO dates, no time-zone shifts on the map slider	Frontend	
Panchayat quantile order <code>p10 <= p50 <= p90</code> everywhere	Backend	
Block-mean of Panchayat values equals block value (consistency test passes)	Backend	
Mock ribbon visible on every screen when <code>data_mode = mock</code>	Frontend	
Empty, loading and error states exist for each screen	Frontend	
Approve on officer screen changes what the farmer app shows	Both	
Farmer feedback tap is stored and visible in database	Both	
All three languages render (fonts loaded, no boxes) on the demo laptop	Frontend	
Audio plays, or fallback to browser speech works offline	Both	
Offline snapshot mode works if the API is killed	Frontend	
Verification numbers on screen equal <code>artifacts/verification.json</code>	Both	

B5. Common integration bugs and fixes

Symptom	Likely cause	Fix
Map polygons appear in the ocean or flipped	Coordinates written <code>[lat, lon]</code>	GeoJSON must be <code>[lon, lat]</code> ; fix in backend export
Chart shows blank line	<code>null</code> values not handled	Frontend filters nulls; backend uses <code>null</code> not <code>NaN</code>
Colors all the same on map	Feature-state values not joined by id	Use <code>promoteId: 'panchayat_id'</code> and matching ids
Panchayat forecast differs from map value	Two different code paths in backend	Both endpoints read the same snapshot table
Types compile locally but fail on other laptop	Stale <code>schema.d.ts</code>	Regenerate types after <code>git pull</code>
API slow (seconds per click)	Model inference on each request	Precompute per issue date into snapshots; API only reads
Hindi or Punjabi shows boxes	Font not loaded or not subset for script	Load Noto Sans Devanagari and Gurmukhi; test offline copy
Advisory approved but farmer app unchanged	Farmer endpoint returns drafts too, or cache	Filter status in <code>(approved, edited)</code> ; invalidate query cache after review

B6. Demo insurance and roles

- **Snapshot mode.** Backend exports contract/snapshot/ (all JSON for the demo issue dates). Frontend can run with `VITE_SNAPSHOT=1` and needs no server at all. If the live API breaks on stage, switch and continue.
- **Two dry runs** at least, timed, with the jury questions list from docs/qna.md (write it together).
- **Roles on stage:** the frontend person drives the screens. The backend person explains the model, the uncertainty and the validation. Both can answer the other's questions in one sentence.
- **Freeze:** no feature work in the last 48 hours. Only fixes.

B7. Demo script (about 6 minutes)

1. Open the map on a monsoon issue date. Show the block forecast as one flat colour. Toggle to Panchayat view. Show the spread and the uncertainty band.
2. Click two Panchayats in the same block. Show different forecasts, then different advisories for the same crop, each with a reason.
3. Open the observed overlay to show what really happened (mock: synthetic truth, say so).
4. Switch to the Review screen. Edit one advisory, approve it. Show the audit trail.
5. Switch to the farmer phone view. Show the approved card in Hindi, play the audio, tap "Did it rain?".
6. Open Verification: table against B0, B1, B2 with confidence intervals, reliability plot, and the honest note about where the model does not help.
7. Open Impact: replay of the spray and heat decisions, model versus block level.
8. Close with the mock-to-real plan: same contract, real files swapped in.